

# 数学に関する質問文分類プログラムの比較検証 - TF-IDF を用いた文章のベクトル化と機械学習の実装 -

3 年 F 組 岡田 浩毅

## 1. 序論

### 1.1 研究動機

近年、世界中の様々な分野において AI の活用が進んでいる。これまで、AI 型チャットボットについていくつかの研究を行ってきた<sup>(1)(2)</sup>が、その中で、高校生として授業を受けている視点から学校外でも質問に答えてくれるチャットボットサービスがあれば便利だと考えるようになった。さらに、実際に調べてみたところ、教育業界でも AI の活用は進んでいる。例として、チャットボットを用いて、学習進度管理、演習授業支援を行う研究が行われている。教育において、AI 型チャットボットの活用は、より良い学習環境を提供することができる。

また、一般に提供されているツールなどを利用することにより、QABot など、チャットボットのシステムを比較的容易に構築することができるようになっている。これまでに、Microsoft Azure や IBM Watson などのサービスを活用してチャットボットについて研究<sup>(1)(2)</sup>を行ってきた。この研究では、チャットボットの効率的な学習方法について模索し、一定の成果を得ることができた<sup>(1)(2)</sup>。さらに、これらの研究成果を発展させるためには、ブラックボックスとなっている IBM Watson を再現することでより効率的な学習方法を見つけることができるのではないかと、という知見を得ることもできた。加えて、この研究成果をもとに文章の類似度比較プログラムを作成し IBM Watson と比較する実証研究を試みたが、単純な文章の類似度比較だけでは再現が難しいことがわかってきた。

そこで、本研究では、単純な文章の類似度比較だけでなく、チャットボットを構築するためのシステムに適した機械学習について模索し、文章のベクトル化と組み合わせることで IBM Watson の再現を目指す。

### 1.2 研究目的

本研究は、チャットボットのブラックボックス解明を目指し、チャットボットを構築するためのシステムに適した機械学習について実証研究を行うことを目的とする。

現在、自然言語処理において日本語は発展途上である。多くの自然言語処理ライブラリは英語を基に開発されている。後に日本語対応モデルが公開される。しかし、日本語の文構造は英語に比べ複雑であり、英語モデルを日本語に完璧に対応させるのは難しい。よって、日本語を基に自然言語処理ライブラリを開発する必要がある。そのため、日本語の自然言語処理ライブラリの開発に伴って、機械学習の手法についても日本語において効率のいい手法を模索する必要がある。

### 1.3 チャットボットの活用事例

教育に関するチャットボットの活用事例を 2 つあげる。1 つ目は、小菅<sup>(3)</sup>らが、REFLECTION-BOT というチャットボットを、個別指導における学習進度管理、復習システムとして活用している。成果として生徒の成績、理解度向上を報告している。2 つ目は、渥美<sup>(4)</sup>らが、SOTARO を開発して、演習授業支援に運用している。

### 1.4 用語解説

#### 1.4.1 IBM Watson<sup>(5)</sup>

IBM Watson はビジネスでの活用の特化した AI サービスの製品群である。顧客サービスのセルフサービス化、コンタクトセンターのオペレーター支援など、言語でのやりとりが大量に発生する業務で活用されている。例として、みずほ銀行や JR 東日本などのお問い合わせセンターの業務支援に用いられている。

#### ● Watson Assistant

IBM Watson の製品群の中の 1 つである。AI の強みを生かして優れた顧客対応を実現する対話型 AI プラットフォームを提供している。IBM Watson について調査した際に用いた。

#### ● 信頼度スコア

IBM Watson を用いて学習した際に、質問ごとに表示される精度のこと。調査の時に、機械学習の精度の指標として利用した。

#### 1.4.2 自然言語処理 (NLP)<sup>(6)</sup>

自然言語とは、人間が日常的に読み書きするのに使用されている言語のことである。日本語や英語、中国語は自然言語の一種である。自然言語処理とは、自然言語を処理する技術や学術分野のことを示す。目的はコンピュータに自然言語を認識させ、人間にとって役立つ様々なタスクを実行させることである。タスクの例として、形態素解析が挙げられる。

#### ● 形態素解析

形態素解析とは、テキストを単語レベルで解析するために、品詞分解する処理のことである。形態素解析を行うためのソフトウェアを形態素解析器と呼び、McCab や Janome が挙げられる。

#### 1.4.3 機械学習<sup>(7)</sup>

機械学習とは、機械に明示的にプログラムすることなく、データから学習させる技術である。データ間の関係性を表現するものをモデルと言い、機械学習ではルールを書くのではなく、モデルを学習させることで自らルールを学習する。機械学習の活用例として、画像認識や音声認識、データ分析が挙げられる。学習法は教師あり学習、教師なし学習、強化学習の三つに分類できる。本研究で用いた学習法は教師あり学習である。

## ● 教師あり学習

教師あり学習とは、入力と出力のペアのデータを使って、ある値が入力された際に適切な値を出力するモデルを学習する手法である。教師あり学習は、分類と回帰に分けられる。分類とは、入力したデータが、予め決められたクラスの中で、どのクラスに属するかを判断する方法である。クラスは、ラベルやターゲットとも言う。分類に用いるデータはラベル付けされている必要があり、このようなデータをラベル付きデータという。分類では、データをグラフ上にプロットし、ラベルのグループの間に決定境界と呼ばれる境界を決定することでデータを分類する。例として、迷惑メールのフィルタリングが挙げられる。回帰とは、データから数値を予測する方法である。回帰も分類と同様にデータをグラフ上にプロットし、回帰直線を引き、数値を予測する。例として、不動産価格の予測を物件の場所や敷地面積、部屋数などから予測する活用事例が挙げられる。

## ● 特徴量エンジニアリング

データの特徴を定量的に表したものを特徴量といい、様々なデータから機械学習に輸入する特徴的データを抽出し1つのデータにまとめることを特徴量エンジニアリングという。機械学習の精度向上には、何を特徴量として機械学習を行うかが重要な要素である。

### 1.4.4 Python<sup>(8)</sup>

Pythonとは1990年代から公開されているプログラミング言語で、読みやすさやわかりやすさを重視したスクリプト言語である。また、実用性と高い拡張性を持ち、機械学習に加えシステム管理やアプリケーション開発などで広く利用されている。Pythonが利用されている代表的なアプリケーションとして、YouTubeやInstagramが挙げられる。今回Pythonを利用した理由は、機械学習を実装する上で多くの機械学習向けのライブラリやフレームワークがPythonに存在するからである。例として、GoogleのTensorFlowやFacebookのPyTorchなど、機械学習に用いる主要なパッケージがある。加えて、簡単に機械学習を実装できるscikit-learnというライブラリが存在する。

## 1.5 環境解説 (Python)

バージョン: 3.11.4

エディター: Visual Studio Code

使用ライブラリ

### 1.5.1 NumPy<sup>(9)</sup>

NumPyはC言語のような計算パフォーマンスをPythonに提供し、処理速度をあげることができるライブラリである。多次元配列を操作する機能と数値計算用の関数が提供されている。

### 1.5.2 Pandas<sup>(10)</sup>

Pandasはデータフレームを扱うことができるライブラリである。CSVファイルやExcelファイルにデータを入力出力できる。機械学習のデータセットを作成する際に用いる。

### 1.5.3 scikit-learn<sup>(11)</sup>

scikit-learnは機械学習ライブラリである。教師あり学習、教師なし学習に関するアルゴリズムが一通り利用でき、サンプルデータセットが豊富に揃っている。今回、scikit-learnの関数を用いてcos類似度とTF-IDFを算出した。

## ● cos 類似度

cos 類似度とは、下式のように2つのベクトル $\vec{x}, \vec{y}$ がなす角 $\theta$ のcosを求めることで、その類似度を判定するものである。

$$\cos\theta = \frac{\vec{x} \cdot \vec{y}}{\|\vec{x}\| \|\vec{y}\|} = \vec{e}_x \cdot \vec{e}_y$$

## ● TF-IDF<sup>(12)</sup>

TFは、索引語頻度と呼ばれる局所的重み付けであり、文書群 $D_n$ 内の特定の文書 $D_j$ における索引語 $\omega_i$ の出現頻度 $f_{ij}$ で定義される。

$$TF(\omega_i, D_j) = f_{ij}$$

IDFは、文書頻度の逆数と呼ばれる大域的重み付けである。文書数をN、索引語 $\omega_i$ を含む文書数を $n_i$ とすると、scikit-learnでは下式で定義される。

$$IDF(\omega_i) = \log \frac{1 + N}{1 + n_i} + 1$$

TF-IDFは、TFとIDFの積のことであり、scikit-learnではこれを正規化した数値が用いられる。

### 1.5.4 Openpyxl<sup>(13)</sup>

Openpyxlは、Excelファイルを入出力することができるライブラリである。Pandasと異なり、新しいExcelファイルを作成することができるので、データを管理するために用いた。

### 1.5.5 Janome<sup>(14)</sup>

Janomeは、形態素解析を行うことができる辞書内包型のライブラリである。他の形態素解析ツールにChasenとMeCabがあるが、精度が同等であり実装が容易であったのでJanomeを用いた。

## 2. 仮説設定

### 2.1 共同研究

#### 2.1.1 概略

2021年から2023年にかけて早稲田大学高等学院の岩村とチャットボットの学習法や内部処理について研究を行った。研究内容は、CIEC春季カンファレンス2021で「Watsonによるチャットボットの学習方法」<sup>(1)</sup>について、CIEC春季カンファレンス2023で「二つの文章に関する類似度スコア計算プログラムの比較検討」<sup>(2)</sup>について発表をした。

#### 2.1.2 Microsoft Azure について

Microsoft Azureの特徴を2つ挙げる。1つ目は、複数の文章を同じ質問として学習を行う際に、助詞・記号・文末などの細かな表現の変更での質問だけでなく、意味を保ったまま、単語の順番や表現を大きく変える必要があること。2つ目は、精度を維持するためには、質問群の数に応じて質問群内の質問数の数を増やす必要があることである。

### 2.1.3 Watson Assistant について<sup>(1)</sup>

Watson Assistant にも Microsoft Azure と同様な特徴が見られた。更に、他に見られた特徴を3つ挙げる。1つ目は、過学習によって精度が落ちること。2つ目は、1つの質問群の内容が Q&A ボット全体の精度に影響を及ぼすことである。

### 2.1.4 類似度計算プログラムについて<sup>(2)</sup>

類似度計算プログラムとは、Watson Assistant の特徴の再現を試みて、独自に作成したプログラムである。2つの文章の類似度を、cos 類似度と TF-IDF で重み付けを用いて算出する。cos 類似度を使用した理由は、共通の単語、表現を避けるべきだという Watson Assistant の特徴から、文章の要素を特徴量として学習を行っていると考えし、単語の頻度をベクトルとして類似度を算出する cos 類似度を使用した。TF-IDF については、過学習による精度の低下、1つの質問群の内容が Q&A ボット全体の精度に影響を及ぼすという特徴から、全質問群とそれぞれの質問群の関係性を信頼度スコアの計算に用いているのではないかと考察し、全文章と該当文章の単語の出現頻度から要素の重み付けが可能な TF-IDF を使用した。結果として、共通の単語によるスコアの上昇を再現することができた。しかし、2つの文章のみで TF-IDF を用いたため、重要な要素である名詞、動詞、形容詞よりも助詞、助動詞の出現回数が少なくなり、重要な要素の重みが小さくなってしまった。このことから Watson Assistant の再現には、質問のデータセットを用いて TF-IDF を算出する必要があると考えた。また、単なる類似度比較だけでは、異なる表現を用いた同義の質問に対応できないため、機械学習の実装の必要性が見受けられた。

## 2.2 機械学習について

### 2.2.1 サポートベクターマシン<sup>(15)</sup>

サポートベクターマシン (SVM) は、マージンの最大化によって決定境界として最適な超平面を学習するアルゴリズムである。マージンとは、サポートベクターという決定境界に最も近いデータから決定境界までの距離である。メリットは、過学習が起りにくいことと、データの次元が大きくなっても識別精度が高いことが挙げられる。デメリットは、学習データが増えると計算量が膨大になることである。

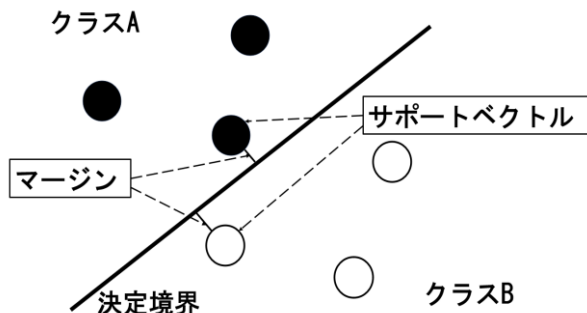


図1 2次元2クラス分類のSVM

### 2.2.2 Naive Bayes<sup>(16)</sup>

Naive Bayes (単純ベイズ分類器) は、ベイズの定理に基づいて各クラスに分類される確率を計算し、最も確率が高いクラスに分類するアルゴリズムである。

$P(B)$ を事象Aが起きる前の事象Bが起きる確率（事前確率）、 $P(B|A)$ を事象Aが起きた後で事象Bが起きる確率（事後確率）としたとき、 $P(B|A)$ は下式のように表せる。これをベイズの定理という。

$$P(B|A) = P(B) \cdot \frac{P(A|B)}{P(A)}$$

Naive Bayes では以下のように計算する。

$$P(y|\vec{x}) = P(y) \cdot \frac{P(\vec{x}|y)}{P(\vec{x})}$$

- $y$  : クラス
- $\vec{x}$  : 分類に使いベクトル
- $P(y|\vec{x})$  :  $\vec{x}$ が含まれるとき  $y$  である確率
- $P(y)$  : 全体のうち、 $y$  である確率

ここで $\vec{x}$ の各要素は互いに独立であると仮定する。

メリットは、とても大きなデータセットに対しても有効なことや、単純で現実世界の多くの複雑な問題に対してうまく機能することが挙げられる。デメリットは、各特徴量が独立であると仮定するの必要があり、実データでは成り立たないことが多いことである。

### 2.2.3 k近傍法<sup>(17)</sup>

k近傍法 (KNN, k-NN) は、周囲のデータを用いて多数決を行うことで推測するアルゴリズムである。具体的には、未知データから近い順にk個のデータを選択し、選択したデータで多数決を取る。図2では $k=5$ として、黒:2、白:3のため白と推測される。メリットは、単純かつ明瞭なアルゴリズムであり、モデルの構築が速い事が挙げられる。デメリットは、局所最適に陥りやすいことである。局所最適とは、ある範囲においての最適解であり、十分な精度が得られない場合がある。

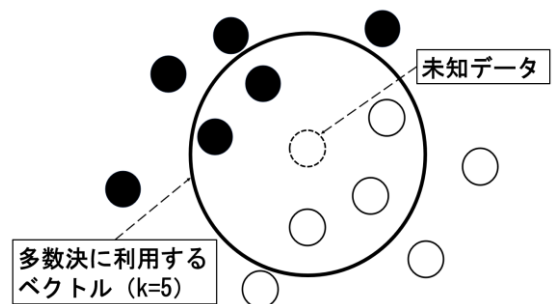


図2  $k=5$ での2クラス分類のk近傍

### 2.2.4 精度比較<sup>(18)</sup>

TP (True Positive, 真陽性) :

予測値を正として、その予測が正しい場合

TN (True Negative, 真陰性) :

予測値を負として、その予測が正しい場合

FP (False Positive, 偽陽性) :

予測値を正として、その予測が誤りの場合

FN (False Negative, 偽陰性) :

予測値を負として、その予測が誤りの場合

- 正解率

$$\text{正解率(Accuracy)} = \frac{TP + TN}{TP + FP + TN + FN}$$

- 適合率

$$\text{適合率(Precision)} = \frac{TP}{TP + FP}$$

- 再現率

$$\text{再現率(Recall)} = \frac{TP}{TP + FN}$$

- F1 値

$$F1 = 2 * \frac{\text{適合率} * \text{再現率}}{\text{適合率} + \text{再現率}}$$

## 2.3 仮説

これまで研究を行ってきた成果をもとに、ここでは、TF-IDF によるベクトル化と SVM による機械学習によって Watson Assistant が再現できるのではないかと、仮説を立てることとする。

この仮説を検証するために、類似度計算プログラムを用いた際に、文章の類似度比較だけでなく機械学習の必要性が見受けられたため機械学習のアルゴリズムについて考え、実証を行う。合わせて、過学習による精度の低下、1つの質問群の内容がチャットボット(Q&A ボット)全体の精度に影響を及ぼすという特徴について考察を行う。

## 2.4 研究方法

### 2.4.1 データセットについて

データセットの作成にあたって、早稲田大学高等学院・中学部の学生を対象に数学クイズ作成の協力をしてもらった。結果として、10個の質問群で計140個の質問の作成ができた。この質問文を形態素解析し、TF-IDF を用いてベクトル化するプログラムを作成する。このプログラムで生成されたベクトル群をデータセットとして機械学習を行う。

#### 二次関数の一般形

|                                                                                                                                                   |                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| Question                                                                                                                                          | Answer                           |
| <ul style="list-style-type: none"> <li>二次関数は、一般的にどのような形で表されるのですか？</li> <li>二次関数を表すための一般的な式の形式は何ですか？</li> <li>二次関数の一般的な表現方法について教えてください。</li> </ul> | $y = ax^2 + bx + c \ (a \neq 0)$ |

⋮

#### 導関数の定義

|                                                                                                                      |                                                          |
|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| Question                                                                                                             | Answer                                                   |
| <ul style="list-style-type: none"> <li>導関数の定義は何ですか？</li> <li>導関数をどのように定義しますか？</li> <li>導関数の定義について教えてください。</li> </ul> | $f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$ |

⋮

図3 Q&A 例

### 2.4.2 機械学習について

機械学習についても、自ら作成したデータセットを用いて機械学習を行うプログラムを作成する。このプログラムを用いてアルゴリズムを変え精度を比較する。精度を測るために、トレーニングデータとテストデータを分割する乱数を固定し、テストデータの正答率を比較する。

### 2.4.3 Watson Assistant との比較について

Watson Assistant との比較については、自ら作成したプログラムに学習させたデータセットを、同様にトレーニングデータとテストデータに分割し、テストデータでの正答率を用いて行う。また、その際に正解できなかった質問についても比較する。

## 3. プログラム作成

### 3.1 分かち書き関数

分かち書きとは、文章を品詞分解し要素ごとに分ける処理のことである。この関数では、質問の文章を引数として、分かち書きを行った文章を返す。具体的には、Janome を用いており、janome.tokenizer パッケージのTokenizer オブジェクトを作り、tokenize() メソッドに解析したい文字列を渡すことで文章解析を行い、形態素情報のうち表層形のみを抽出して使用した。この関数は、文章をベクトル化する際に分かち書きをする必要があるため、その際に使用している。また、データフレーム作成の際に使用する辞書リストの作成を同時に行っている。

```
def wakatigaki(text):
    t = Tokenizer()
    tokens = t.tokenize(text)
    docs=[]
    for token in tokens:
        docs.append(token.surface)
    global dictionary
    dictionary.append(docs)
    return docs
```

### 3.2 文章ベクトル化関数

この関数では、全文章を引数として、全文章をベクトル化した行列を返す。ベクトル化の方法は、文章内の単語の出現回数をベースとして、TF-IDF による重み付けの処理を行った。ベクトル化には scikit-learn のTfidfVectorizer を用いており、分かち書き関数を使用し、品詞分解した要素単位で特徴量を生成した。

```
def vectorization(documents):
    docs = np.array(documents)
    vectorizer = TfidfVectorizer(
        analyzer=wakatigaki,
        binary=True, use_idf=True)
    vecs = vectorizer.fit_transform(docs)
    return vecs.toarray()
```

### 3.3 データフレーム作成

機械学習を行うにあたって、ベクトル化した文章にラベルを付け、データフレーム化する必要がある。このプログラムでは、分かち書きした全文章から作成した辞書リストを列名とし、ベクトルをデータフレーム化した。

```
headers.extend(sorted(set(sum(dictionary, []))))
)
datas=vectorization(q_list())
df=pd.DataFrame(
    data=datas, index=indexs, columns=headers)
df.to_csv("dataset.csv")
```

### 3.4 機械学習用データセット作成

scikit-learn の fit 関数は引数に Bunch オブジェクトを用いるため、データフレームを Bunch オブジェクトに格納した。また、機械学習にあたって目的変数と説明変数を設定し、特徴量にデータフレームの列名とした辞書リストを入れた。学習、精度の判定のためにトレーニングデータとテストデータに分割したが、他のアルゴリズムと比較する際に分け方を変化させないために乱数を 10 に固定した。

```
qnot = sklearn.utils.Bunch()
qnot['target'] = df['label']
qnot['data'] = df.iloc[:, 1:]
qnot['feature_names'] = df.columns.values[1:]
# トレーニングデータとテストデータに分割
X_train, X_test, y_train, y_test=train_test_split(
    qnot['data'],
    qnot['target'],
    random_state=10)
```

### 3.5 機械学習の実装

機械学習のアルゴリズムごとの精度を比較するために、各アルゴリズムのモデルを構築し、学習を行った。精度比較については、テストデータでの正答率、適合率 (macro avg)、再現率 (macro avg)、F1 値 (macro avg) を出力した。k 近傍法については近傍とする最適なデータ数を求めるために、近傍の数を 1 から 100 まで変更していきグラフをプロットした。結果、最適な近傍の数は 16 であった。

```
accuracy_list = []
k_range = range(1, 100)
for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    Y_pred = knn.predict(X_test.values)
    accuracy_list.append(
        metrics.accuracy_score(
            y_test, Y_pred))
```

```
figure = plt.figure()
ax = figure.add_subplot(111)
ax.plot(k_range, accuracy_list)
ax.set_xlabel('k-nn')
ax.set_ylabel('accuracy')
plt.show()
```

```
#SVM
svm = SVC(kernel='rbf', random_state=None)
svm.fit(X_train, y_train)
svm_predicted = svm.predict(X_test)
print(classification_report(
    y_test, svm_predicted, digits=5))
# Naive Bayes
gnb = GaussianNB(
    priors=None, var_smoothing=1e-09)
gnb.fit(X_train, y_train)
gnb_predicted = gnb.predict(X_test)
print(classification_report(
    y_test, gnb_predicted, digits=5))
# K 近傍法
knn = KNeighborsClassifier(n_neighbors=16)
knn.fit(X_train, y_train)
knn_predicted = knn.predict(X_test.values)
print(classification_report(
    y_test, knn_predicted, digits=5))
```

## 4. 結果・分析

### 4.1 データセット作成

データセット作成に際して行った文章ベクトル化の実行結果として、140 個の質問文を、それぞれ 203 次元の単位ベクトルに変換することができた。また、これらの辞書とベクトルを基に 140 行×203 列のデータフレームを生成し、このデータフレームを基にデータセットを作成することができた。

### 4.2 精度比較

各アルゴリズムと Watson Assistant でのテストデータを用いた精度は表 1 のようになった。最も正答率が高かったアルゴリズムは SVM、K 近傍法であり、最も正答率が低かったアルゴリズムは Naive Bayes であった。適合率、再現率、F1 値について比較すると、SVM よりも K 近傍法のほうが高くなっている。Watson Assistant と比較すると、SVM と K 近傍法は Watson Assistant をすべての値で上回っている。

表 1. 各アルゴリズムの精度

|             | 正解率     | 適合率<br>(macro) | 再現率<br>(macro) | F1 値<br>(macro) |
|-------------|---------|----------------|----------------|-----------------|
| SVM         | 0.97143 | 0.96667        | 0.97500        | 0.96571         |
| Naïve Bayes | 0.88571 | 0.90167        | 0.89667        | 0.88127         |
| K 近傍        | 0.97143 | 0.97500        | 0.97500        | 0.97143         |
| Watson      | 0.94286 | 0.95000        | 0.94167        | 0.93662         |



#### 4.3 間違えた質問

間違えた質問は表2のようになった。全てに共通して「SSS, SAS, ASA がそれぞれ何を示しているか教えて」を間違えている。しかし、回答ラベルは全て異なっている。また、間違えた質問の回答ラベルに法則を見出すことができなかった。

表2. 各アルゴリズムで間違えた問題

|             | 間違えた質問                         | 回<br>答 | 正<br>解 |
|-------------|--------------------------------|--------|--------|
| SVM         | SSS, SAS, ASA がそれぞれ何を示しているか教えて | 6      | 3      |
|             | SSS, SAS, ASA がそれぞれ何を示しているか教えて | 9      | 3      |
| Naive Bayes | 三角形の合同条件をすべて書きなさい。             | 5      | 3      |
|             | 共通因数とは何か簡潔に答えて？                | 7      | 4      |
|             | 三平方の定理を式とともに説明せよ               | 10     | 5      |
| K 近傍        | SSS, SAS, ASA がそれぞれ何を示しているか教えて | 1      | 3      |
| Watson      | SSS, SAS, ASA がそれぞれ何を示しているか教えて | 7      | 3      |
|             | 実数解を持つ二次関数の解の存在確認の方法を教えて       | 1      | 9      |

#### 4.4 Watson Assistant を用いた際の信頼度スコア

Watson Assistant の精度をより詳しく測るために各質問の信頼度スコアを比較した。表3に挙げた質問以外は信頼度スコアが0.8以上であった。「SSS, SAS, ASA がそれぞれ何を示しているか教えて」については0.13246というかなり低い信頼度スコアを取ったが、「実数解を持つ二次関数の解の存在確認の方法を教えて」については、間違えているが0.63252という低い信頼度スコアを取っている。

表3. Watson Assistant を用いた時に0.8以下の信頼度を取った質問

| 質問                             | 回答 | 正解 | 信頼度<br>スコア |
|--------------------------------|----|----|------------|
| abc を用いて解の公式を表せ                | 1  | 1  | 0.64796    |
| SSS, SAS, ASA がそれぞれ何を示しているか教えて | 7  | 3  | 0.13246    |
| 実数解を持つ二次関数の解の存在確認の方法を教えて       | 1  | 9  | 0.63252    |

### 5. 考察

#### 5.1 共通で間違えた質問について

全てのアルゴリズム、Watson Assistant で「SSS, SAS, ASA がそれぞれ何を示しているか教えて」（以下 Q“SSS”とする）という質問に間違えた理由を考える。「SSS」、「SAS」、「ASA」は英語で三角形の合同条件を示す単語であるが、今回用いた質問リストには

Q“SSS”以外に「SSS」、「SAS」、「ASA」を用いた質問は存在しなかった。そのため、乱数を10に固定した際に Q“SSS”がテストデータに分割され、「SSS」、

「SAS」、「ASA」について学習が行われなかった。表3より、Watson Assistant を用いた時に Q“SSS”では、回答ラベル「7」で信頼度スコア0.13246であった。これは、Q“SSS”がラベル「1」～「10」の質問群の内どれとも似ていないということを示しており、「SSS」、「SAS」、「ASA」について学習が行われなかった結果である。他のアルゴリズムでも、Watson Assistant と同様にラベル「1」～「10」の質問群の内どれとも似ていないと判断し、正解ラベル「3」を出力できなかったと考える。全てのアルゴリズムで Q“SSS”の回答ラベルが異なる理由については、それぞれのアルゴリズムでの分類方法が異なるためであると考えられる。

#### 5.2 SVM と k 近傍法の過学習の可能性について

SVM と k 近傍法での過学習の可能性について考察する。今回のデータセットでは、同じような質問文が多く存在していたため、過学習に陥っていても判断しづらくなっている。SVM については、2.2 で述べたように過学習に陥りづらいというメリットがあるため、過学習だとは考えづらい。対して、k 近傍法は近傍 (k) の数を小さくすると過学習に陥りやすく、大きくすると精度が低くなるという特徴がある。図4より、最も正答率が高いのは k=16 であるが過学習の可用性が考えられる。今回の検証では、精度と過学習のバランスの検証を行うことができなかった。

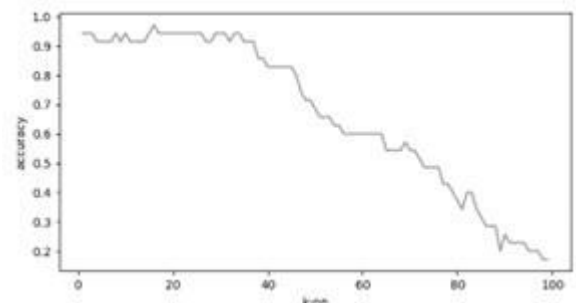


図4 kを1から100まで変化させたときの正答率推移

#### 5.3 今回のケースに適する機械学習のアルゴリズム

表1より、今回作成したデータセットについては K 近傍法が最も精度が良い。しかし、SVM のほうが K 近傍法に比べて実装に適していると考えられる。その理由は二つ挙げられる。一つ目は、K 近傍法と SVM の正答率、適合率、再現率、F1 値はどちらも高く、とても近い値を取っているためである。二つ目は、K 近傍法では最適な近傍の数を求める工程が必要になるためである。K 近傍法では、精度と過学習のバランスを考慮する必要があり、プログラムでの処理が増える。精度と実装の容易さの両者を総合的にみると、SVM が優れていると考えられる。

#### 5.4 SVM の決定境界の可視化

トレーニングデータに用いた 203 次元のベクトル化した文章を二次元に圧縮し、散布図としてプロットすることで、SVM での決定境界の可視化を試みた。次元圧縮では主成分分析を用いた。主成分分析とは、多次元のデータをできるだけ特徴を損なわずに低次元空間に圧縮する方法である。二次元に圧縮した際の散布図は図 5 のようになった。しかし、表 4 より、精度が顕著に落ちていることが分かり、図 5 を基に 203 次元での SVM の決定境界を考察することはできないと考える。今回の検証での SVM の決定境界の可視化には、主成分分析以外の手法を用いる必要があると考えられる。

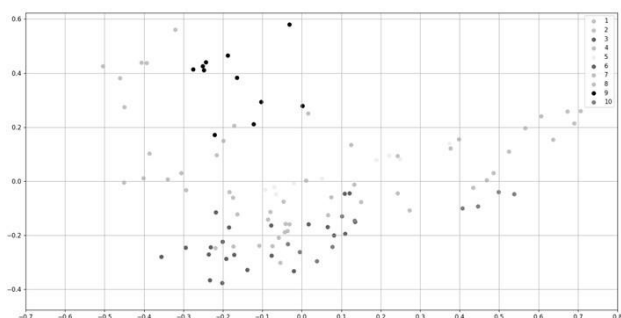


図 5 トレーニングデータを 2 次元に圧縮した散布図

表 4. 2 次元に圧縮した際の SVM の精度

|           | 正解率     | 適合率<br>(macro) | 再現率<br>(macro) | F1 値<br>(macro) |
|-----------|---------|----------------|----------------|-----------------|
| トレーニングデータ | 0.66667 | 0.71419        | 0.66025        | 0.64041         |
| テストデータ    | 0.08571 | 0.10333        | 0.08333        | 0.08889         |

#### 5.5 Watson Assistant との比較

仮説について精度の面でいえば、SVM と K 近傍法では Watson Assistant と同じレベルのモデルを構築することができた。しかし、間違えた問題の間違え方が異なる点や、信頼度スコアと同様な質問ごとのスコアを導入することができなかったため、再現できたとはいえないと考える。今回のモデルは実際の運用を考えておらず、ストップワード、データ量の増加に対応する必要があると考えられる。データ量が増加した際には、TF-IDF によるベクトル化で 203 次元以上のベクトルの計算処理を行う必要があり、処理時間の増加を考慮する必要がある。このとき、TF-IDF によるベクトル化は最適といえない可能性があり、Watson Assistant では TF-IDF によるベクトル化を行っているとは断定できないため、再現のためには他の手法を検討する必要がある。

#### 6. 結論

本研究では、チャットボットのブラックボックス解明を目指し、チャットボットを構築するためのシステムに適した機械学習について研究を行なった。具体的には、Watson Assistant は TF-IDF によるベクトル化と SVM による機械学習で再現することができるのではないかと、いう仮説を立て、その仮説を検証するための実証研究を行なった。

その結果、文章のベクトル化の面では TF-IDF を用いて、単語の出現頻度から要素の重み付けを行ったデータセットの作成に成功した。機械学習の面では、SVM と K 近傍法は Watson Assistant 以上の精度を出し、Naïve Bayes は他に比べて精度が低かった、ということがわかった。また、どのアルゴリズムでも、新出の要素が登場する場合には間違えてしまうことは当然だが、今回の実証研究では、間違い方が異なることも新たに確認できた。この間違い方が異なる要因は各アルゴリズムの分類方法が異なることに起因する。加えて、SVM と K 近傍法については過学習の可能性を考察したが、アルゴリズムの特徴上 SVM は過学習していると考えられ、K 近傍法については新たに過学習と正答率のバランスが取れる近傍の数の検証を行う必要が見られた。これらから、今回のラベル付きデータを用いて、自ら 3 種類以上に分類する機械学習を実装する際には、精度の高さと実装の容易さから SVM が最も優れていると結論づけられる。SVM の決定境界の可視化については、主成分分析を行い二次元に圧縮することでのグラフへのプロットを試みたが、圧縮した際の SVM の精度が極端に落ちたため、別の手法を模索する必要がある。

以上のことから、本論文で立てた仮説の検証結果としては、精度の面で TF-IDF を用いたベクトル化と SVM、K 近傍法による機械学習がチャットボットを構築するためのシステムに適しているといえる。また、本論文の実証研究による検証により、目指している IBM Watson の再現に近づくことができた。今後の課題として、運用を想定すると計算量やストップワードの問題が生じるため更に検証する必要がある、という点などを挙げることができる。

#### 7. 展望

##### 7.1 ストップワードの実装

ストップワードとは、文章をベクトル化する際に、分かち書きした文章から取り除く要素のことである。例として、記号や語尾などが挙げられる。記号や語尾は、文章に大きな意味を与えない。そのため、記号や語尾の類似が特徴量として判断されると、分類の精度が下がってしまう。今回のプログラムでは、TF-IDF による重み付けを行ったが、アンケートによるデータセットの作成を行ったため、記号や語尾は質問グループを跨いで似る傾向が見られた。よって、記号や語尾をストップワードとして設定する必要があると考察できる。

##### 7.2 データ量の増加

今回の検証では、10 個の質問群で計 140 個の質問を用いたが、実際に運用する際にはより多くの質問を用いる必要が出てくると考えられる。また、今回の質問群にはそれぞれの関連性が低く、未対応の質問に対する対応が難しくなっている。そのため、より多くの質問群を用意し、質問群に関連性を見出すことで未対応の質問に対応することができるプログラムの作成を試みたい。

##### 7.3 単語ベクトルの実装

単語ベクトルとは、分かち書き後の文章の要素に意味を与えたものである。単語をベクトル化する技術を

Word2Vec という。例として、「Queen」の単語ベクトルを式として表すと、「Queen」=「王様」-「男性」+「女性」+「英語」となる（日本語を基準とした場合）。単語ベクトルを実装した場合、文章の意味による分類を行うことができる。

#### 参考文献

- (1) 岩村陸, 岡田浩毅:「Watson によるチャットボットの学習方法」, CIEC 春季カンファレンス論文集, 12 , pp.136-137 (2021)
- (2) 岩村陸, 岡田浩毅:「二つの文章に関する類似度スコア計算プログラムの比較検討」, CIEC 春季カンファレンス論文集, 14 , pp.76-77 (2023).
- (3) 小菅李音, 高木正則, 市川尚:「チャットボットと個別指導を併用した数学教育における理解困難箇所の学習支援の実践と評価」, 情報教育シンポジウム論文集, 2020, pp.31-38 (2020).
- (4) 渥美雅保, 村田祐樹, 安川葵:「オープンチャットとロボットの連係によるティーチングアシスタントとの協働システム」, 人工知能学会全国大会論文集 31, (2017).
- (5) IBM Watson : <https://www.ibm.com/jp-ja/watson>, (2023 年 10 月 24 日閲覧).
- (6) 中山光樹:「機械学習・深層学習による自然言語処理入門-scikit-learn と TensorFlow を使った実践プログラミング-」, pp2-4, マイナビ出版, (2020)
- (7) 同上, pp14-19
- (8) Python Japan : <https://www.python.jp/pages/about.html>, (2023 年 10 月 24 日閲覧).
- (9) NumPy : <https://numpy.org/ja/>, (2023 年 10 月 24 日閲覧).
- (10) Pandas : <https://pandas.pydata.org/>, (2023 年 10 月 24 日閲覧).
- (11) Scikit-learn : <https://scikit-learn.org/stable/>, (2023 年 10 月 24 日閲覧).
- (12) 北研二, 津田和彦, 獅々堀正幹:「情報検索アルゴリズム」, pp.33-45, 共立出版 (2002).
- (13) OpenPyXL : <https://openpyxl.readthedocs.io/en/stable/>, (2023 年 10 月 24 日閲覧).
- (14) Janome : <https://moco-beta.github.io/janome/>, (2023 年 10 月 24 日閲覧).
- (15) MathWorks : <https://jp.mathworks.com/discovery/support-vector-machine.html>, (2023 年 10 月 24 日閲覧).
- (16) 古宮嘉那子, 伊藤裕佑, 佐藤直人, 小谷善行:「文書分類のための Negation Naive Bayes」, 自然言語処理 20 (2), pp.161-182 (2013).
- (17) IBM : <https://www.ibm.com/jp-ja/topics/knn>, (2023 年 10 月 24 日閲覧).
- (18) 環境省 : [https://www.renewable-energy-potential.env.go.jp/RenewableEnergy/dat/report/r03-01/r03-01\\_chpt3\\_Part2.pdf](https://www.renewable-energy-potential.env.go.jp/RenewableEnergy/dat/report/r03-01/r03-01_chpt3_Part2.pdf), (2023 年 10 月 24 日閲覧).
- (19) 関栗, 和喜多美月, 杉本理:「Watson Assistant による Web 型チャットボットの開発-城西大学経営学部におけるチャットボットの実装について-」, 城西情報科学研究, pp27-37, 城西大学情報科学研究センター (2022).
- (20) 山内一将, 川本淳平, 堀良彰, 櫻井幸一:「機械学習を用いたセッション分類による C&C トラフィック抽出」, 暗号と情報セキュリティシンポジウム, p.4C1-5, (2014)
- (21) 中山研一朗, 正田備也:「BERT を用いた分類モデルによる冗長な質問文の要点抽出」, 第 21 回情報科学技術フォーラム (2021).
- (22) 東中竜一郎, 稲葉通将, 水上雅博:「Python でつくる対話システム」, オーム社, 2020
- (23) 田中穂積:「自然言語処理-基礎と応用-」, 電子情報通信学会, 1999